

Binary Search versus Linear Search: An Empirical Comparative Analysis of Search Efficiency Across Different Dataset Sizes

Sai Ganesh Mandhati
mandhatisaiganesh@gmail.com

Shlok Uprit
shlokuprit@gmail.com

Department of CSE (AI & ML), Marwadi University

Abstract—The computational complexity of fundamental search algorithms is established knowledge in computer science: Linear Search requires $O(n)$ time while Binary Search operates in $O(\log n)$ time on ordered sequences. However, theoretical asymptotic bounds do not fully characterize practical wall-clock execution time on modern hardware, where constant factors, memory access patterns, and preprocessing overhead significantly influence observed performance. This paper presents an empirical investigation evaluating iterative Linear Search and iterative Binary Search across seven orders of magnitude ($N = 10$ to 10,000,000 elements) on an Apple M1 ARM64 architecture. Using a dual-methodology framework comprising 98,000 high-precision trial observations under batched amortized timing and single-shot baseline measurements, we evaluate positional sensitivity across six target locations, query success/failure asymmetry, and the amortization of `std::sort` preprocessing overhead. Empirical results show that for small sequences ($N \leq 100$), Linear Search is competitive or up to $1.52\times$ faster than Binary Search (7.66 ns vs. 11.64 ns at $N = 10$). In the asymptotic regime ($N = 10^7$), Binary Search delivers a measured speedup of $6,198.5\times$ for average-case queries (260.99 ns vs. 1.62 ms) and $91,425.0\times$ for absent queries. The empirical break-even query count for amortizing sorting preprocessing remained within approximately 114–140 queries across the evaluated dataset sizes ($N \geq 1,000$) under the specified experimental conditions. These findings provide empirically grounded observations relevant to search algorithm selection, subject to the hardware and workload conditions evaluated.

Index Terms—Linear Search, Binary Search, Empirical Algorithmics, Cache Locality, Computational Complexity, Performance Benchmarking, Amortization Analysis.

I. Introduction

Searching represents one of the most fundamental operations in computational systems, underpinning relational database index scans, compiler symbol resolution, in-memory key-value lookups, and operating system kernel dispatch tables [1], [2]. Theoretical computer science classifies search efficiency through asymptotic time complexity: Linear Search exhibits worst- and average-case time complexity of $\Theta(n)$, whereas Binary Search achieves logarithmic time complexity of $\Theta(\log n)$ on strictly sorted collections [3], [4].

While asymptotic notation ($\mathcal{O}$, Ω , Θ) provides rigorous upper and lower bounds as problem size $n \rightarrow \infty$,

it deliberately suppresses constant factors, implementation overhead, memory access latency, and architectural characteristics of the host platform [5], [6]. On modern superscalar microprocessors, the memory hierarchy (L1, L2, system cache, and DRAM), hardware prefetching, and branch prediction may significantly influence observed execution times in ways that asymptotic analysis does not predict [7], [8]. Prior literature suggests that sequential memory scans benefit from contiguous cache-line loading and predictable branching, whereas logarithmic bisections generate non-sequential address jumps that frequently incur additional memory latency [9].

Furthermore, Binary Search strictly requires sorted input data. In dynamic systems where data arrives in an unsorted stream, evaluating Binary Search in isolation obscures the non-trivial computational cost of sorting ($O(n \log n)$) [10], [11]. The practical superiority of Binary Search is therefore not unconditional; it depends critically on:

- 1) The scale of the dataset N ,
- 2) The target element's positional distribution,
- 3) The frequency of repeated queries K executed over the data structure, and
- 4) The timer resolution and benchmarking methodology employed.

To bridge the gap between textbook theory and physical execution, this paper presents an empirical study comparing iterative Linear Search and iterative Binary Search. Rather than assuming the superiority of Binary Search a priori, we systematically quantify the boundary conditions under which each algorithm is optimal.

A. Research Contributions

The specific contributions of this study are:

- **Multi-Scale Empirical Benchmarking:** We execute 98,000 trial-level search observations across seven orders of magnitude ($N = 10$ to 10,000,000), recording execution latency with monotonic nanosecond clocks and hardware-counter characterization.
- **Positional Sensitivity Mapping:** We evaluate performance across six distinct target locations (Beginning,

Quarter, Middle, Three-Quarter, End, and Absent) to isolate $O(1)$, $O(n)$, and $O(\log n)$ behavioral regimes.

- Preprocessing Amortization Analysis: We empirically measure `std::sort` overhead and determine the query threshold K^* required to justify sorting unsorted data for binary search.
- Measurement Methodology Characterization: We contrast single-shot timing with batched amortized timing, establishing how timer quantization (41.67 ns Darwin timebase ticks) and timer overhead (18.85 ns) distort sub-microsecond algorithmic benchmarks.
- Open Scientific Reproducibility Package: All C++ source code, raw observation logs, statistical scripts, and figures are provided in an independently verifiable research package.

II. Related Work

The formal study of searching and sorting algorithms originated in early computing literature. Knuth [1] provided comprehensive mathematical foundations for comparison-based searching, demonstrating that Binary Search is asymptotically optimal within the comparison tree model, requiring at most $\lfloor \log_2 n \rfloor + 1$ comparisons for successful searches. Bentley [4] highlighted subtle implementation pitfalls in binary search, including integer overflow in midpoint calculations ($\text{mid} = \text{low} + (\text{high} - \text{low})/2$).

In empirical algorithmics, McGeoch [5], Johnson [6], and Bailey [12] established formal methodologies for experimental computer science, emphasizing that empirical studies must control for compiler optimizations, cache effects, statistical variance, and avoid selective reporting. Mytkowicz et al. [13] demonstrated that architectural factors, such as link order and environment variable size, can bias microprocessor benchmarks by up to 15% if not systematically isolated.

The role of memory hierarchy in algorithmic performance has been explored extensively by Hennessy and Patterson [7], Drepper [8], and Vitter [14]. In cache-conscious systems, sequential memory access exhibits spatial locality, fetching entire cache lines (typically 64 or 128 bytes) in a single burst. In contrast, Binary Search exhibits poor spatial locality during initial bisections, causing compulsory cache line loads [15], [16].

Sorting algorithms, particularly Introsort (Musser [11]) which combines Quicksort [10] and Heapsort, have established that preprocessing costs scale as $\Theta(n \log n)$. However, the empirical break-even query threshold K^* across dataset scales in modern architectures remains rarely quantified in literature. This study systematically unifies these dimensions.

III. Theoretical Background

A. Linear Search

Iterative Linear Search inspects elements sequentially from index $i = 0$ to $n - 1$ until a matching key is identified or the collection is exhausted.

```

1: function LinearSearch( $A, n, \text{target}$ )
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:   if  $A[i] = \text{target}$  then
4:     return  $i$ 
5:   end if
6: end for
7: return  $-1$ 

```

B. Binary Search

Iterative Binary Search operates on a sorted array A by maintaining boundary indices $[\text{low}, \text{high}]$ and bisecting the candidate range at each iteration.

```

1: function BinarySearch( $A, n, \text{target}$ )
2: low  $\leftarrow 0$ 
3: high  $\leftarrow n - 1$ 
4: while low  $\leq$  high do
5:   mid  $\leftarrow \text{low} + (\text{high} - \text{low})/2$ 
6:   if  $A[\text{mid}] = \text{target}$  then
7:     return mid
8:   else if  $A[\text{mid}] < \text{target}$  then
9:     low  $\leftarrow \text{mid} + 1$ 
10:  else
11:    if mid = 0 then
12:      break
13:    end if
14:    high  $\leftarrow \text{mid} - 1$ 
15:  end if
16: end while
17: return  $-1$ 

```

C. Complexity Comparison

Table I summarizes the asymptotic time and auxiliary space complexities of both algorithms.

TABLE I
Theoretical Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space
Linear Search	$\mathcal{O}(1)$	$\Theta(n)$	$\Theta(n)$	$\mathcal{O}(1)$
Binary Search	$\mathcal{O}(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\mathcal{O}(1)$

D. Preprocessing and Repeated Queries

When input data is initially unsorted, Binary Search requires an antecedent sorting step ($\mathcal{O}(n \log n)$). For a workload of K independent search queries, the cumulative execution cost is modeled as:

$$\text{Workload}_{\text{Linear}}(K) = K \cdot T_{\text{Linear}} \quad (1)$$

$$\text{Workload}_{\text{Binary}}(K) = T_{\text{Sort}} + K \cdot T_{\text{Binary}} \quad (2)$$

The theoretical and empirical break-even query threshold K^* occurs when cumulative costs equate:

$$K^* = \frac{T_{\text{Sort}}}{T_{\text{Linear}} - T_{\text{Binary}}} \quad \text{for } T_{\text{Linear}} > T_{\text{Binary}} \quad (3)$$

IV. Research Questions and Hypotheses

This investigation addresses eight core research questions (RQs):

- RQ1: How does execution time scale with dataset size N for Linear Search and Binary Search?
- RQ2: How does target position influence latency across algorithms?
- RQ3: How do successful and unsuccessful searches differ empirically?
- RQ4: How does sorting preprocessing affect practical search efficiency?
- RQ5: How many repeated queries K^* are required before sorting becomes worthwhile?
- RQ6: How sensitive are performance measurements to timing methodology?
- RQ7: How closely do comparison counts correlate with wall-clock time?
- RQ8: Which findings are universal versus architecture-specific?

We formulate the following formal hypotheses:

- H1: Linear Search empirical latency grows strictly linearly ($\Theta(N)$) while Binary Search scales logarithmically ($\Theta(\log N)$).
- H2: For small sequences ($N \leq 100$), Linear Search may be faster or competitive with Binary Search because constant factors and implementation overhead can dominate over asymptotic advantages.
- H3: For targets at the array beginning ($i = 0$), Linear Search outperforms Binary Search across all N .
- H4: Sorting preprocessing renders Binary Search disadvantageous for small query workloads ($K < K^*$).
- H5: The empirical break-even count K^* stabilizes across large N rather than growing indefinitely.
- H6: Single-shot timing introduces quantization artifacts that distort sub-100 ns measurements.

V. Experimental Methodology

A. Experimental Environment

Benchmarks were executed on an Apple Silicon platform with full system profiling as detailed in Table II.

TABLE II
Host Hardware and Software Environment

Parameter	Host Specification
CPU Model	Apple M1 (8 cores: 4P Firestorm + 4E Icestorm)
Architecture	ARM64 (aarch64, 64-bit little-endian)
L1 Cache (P-core)	192 KB Instruction, 128 KB Data per core
L2 Cache (P-cluster)	12 MB Unified Shared
L1 Cache (E-core)	128 KB Instruction, 64 KB Data per core
L2 Cache (E-cluster)	4 MB Unified Shared
Cache Line Size	128 bytes
System RAM	8.00 GB Unified LPDDR4X
Operating System	macOS 26.6.2 (Darwin Kernel 25.6.0)
Compiler	Apple clang version 21.0.0 (ARM64)
Optimization Flags	-O3 -std=c++17 -Wall -Wextra
Timer Interface	std::chrono::steady_clock (24 MHz hardware timebase)

B. Dataset Construction

Datasets were generated across seven orders of magnitude: $N \in \{10, 10^2, 10^3, 10^4, 10^5, 10^6, 10^7\}$. Elements are 32-bit signed integers (`int32_t`, 4 bytes). Memory footprints range from 40 bytes ($N = 10$) to 38.15 MB ($N = 10^7$), spanning on-chip L1/L2 caches to main memory. Elements follow the monotonic relation $A[i] = (i+1) \cdot 2$, guaranteeing distinct even values. Absent targets were set to $2N + 1$.

C. Query Workloads

Seven experimental conditions were evaluated:

- 1) Beginning: Target at index $i = 0$.
- 2) Quarter: Target at index $i = \lfloor N/4 \rfloor$.
- 3) Middle: Target at index $i = \lfloor N/2 \rfloor$.
- 4) Three-Quarter: Target at index $i = \lfloor 3N/4 \rfloor$.
- 5) End: Target at index $i = N - 1$.
- 6) Absent: Guaranteed non-existent key ($2N + 1$).
- 7) Random Success: Uniform random index $i \sim \mathcal{U}(0, N - 1)$ sampled per trial (seed 1337).

D. Benchmarking Procedure and Anti-Optimization

To prevent compiler dead-code elimination and constant folding under -O3, all search indices were accumulated into a sink variable passed to an inline assembly barrier:

```
asm volatile("" : "+r,m"(val) : : "memory");
```

Each condition block was preceded by 100 untimed warmup iterations. In Benchmark Version 2, execution order was interleaved ($t \pmod 2$) to eliminate systematic thermal and cache residency bias.

E. Timing Methodology

The host timer interface (`std::chrono::steady_clock`) derives from the ARM64 CNTVCT_EL0 register operating at 24 MHz (41.67 ns tick resolution). Measured back-to-back timer invocation exhibits a median overhead of 18.85 ns. To achieve high measurement fidelity, Benchmark Version 2 utilizes batched amortized timing:

$$T_{\text{query}} = \frac{T_{\text{batch}}}{B} \quad (4)$$

where batch size $B \in [2, 5000]$ was dynamically calibrated to ensure each measured block spans $> 50 \mu\text{s}$ to several milliseconds.

F. Comparison Counting

Following principles of software instrumentation [17], key comparisons are instrumented separately from timed executions to eliminate measurement overhead:

- Linear Search: Exactly 1 comparison ($A[i] = \text{target}$) per inspected element.
- Binary Search: 3-way branching: 1 comparison for equality ($A[\text{mid}] = \text{target}$) plus 1 comparison for ordering ($A[\text{mid}] < \text{target}$) upon mismatch.

G. Statistical Analysis

For all 98,000 trials, we compute sample mean ($\bar{x}$), sample median ($\tilde{x}$), sample standard deviation (s), and 95% confidence intervals using Student's t -distribution with $n - 1 = 999$ degrees of freedom ($h = t_{0.975,999} \cdot \frac{s}{\sqrt{n}}$).

VI. Experimental Results

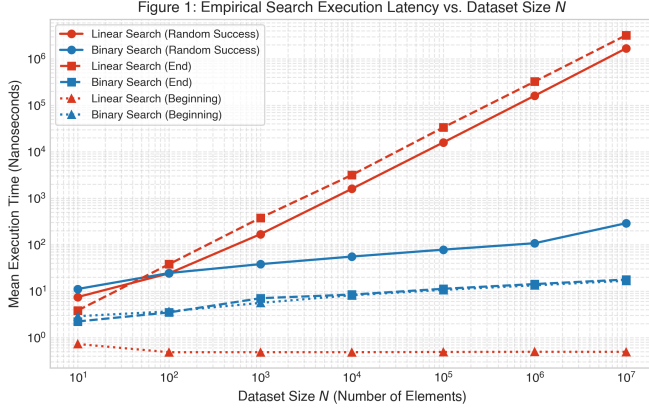


Fig. 1. Empirical search execution latency vs. dataset size N on log-log scale for Linear Search and Binary Search across representative conditions.

A. Dataset-Size Scaling and Execution Latency

Figure 1 and Table III present the primary latency scaling results for Version 2 batched measurements.

At $N = 10$, Linear Search outperforms Binary Search by $1.52\times$ (7.66 ns vs. 11.64 ns). The two algorithms reach approximate parity at $N \approx 100$ (26.00 ns vs. 26.16 ns). Beyond $N = 100$, Binary Search demonstrates increasing speedup ratios, reaching $6,198.52\times$ at $N = 10^7$ for average-case queries and $91,424.95\times$ for absent queries.

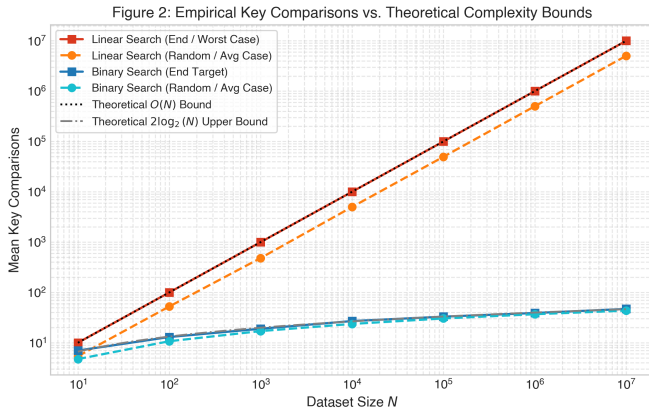


Fig. 2. Empirical key comparisons vs. theoretical $O(N)$ and $2\log_2 N$ bounds across dataset sizes.

B. Comparison-Count Analysis

Figure 2 contrasts measured key comparisons with theoretical bounds. For Linear Search, comparisons scale exactly as N for absent/end queries and $N/2$ for random queries ($R^2 = 1.0000$). For Binary Search, comparisons follow the logarithmic envelope, executing 4.74 comparisons at $N = 10$ and 43.51 comparisons at $N = 10^7$, strictly adhering to the theoretical upper bound $2\lceil\log_2 N\rceil + 2$.

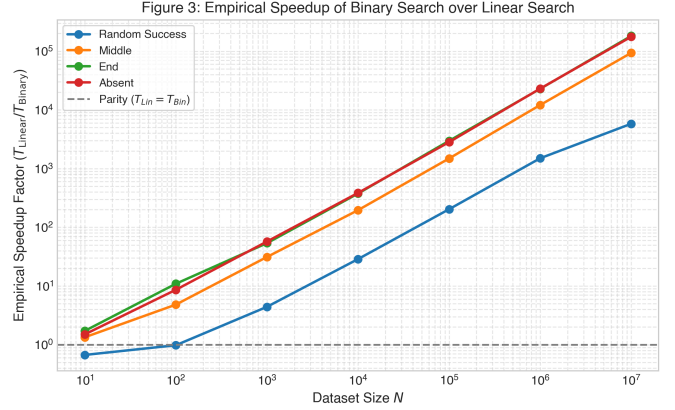


Fig. 3. Empirical speedup factor ($T_{\text{Linear}}/T_{\text{Binary}}$) across search conditions.

C. Relative Speedup Dynamics

Figure 3 illustrates the empirical speedup ratio across conditions. Speedup diverges rapidly for $N \geq 1,000$. For absent queries, the speedup reaches nearly five orders of magnitude ($91,425\times$) at $N = 10^7$, as Linear Search scans the entire 38.15 MB buffer while Binary Search terminates in 48 comparisons.

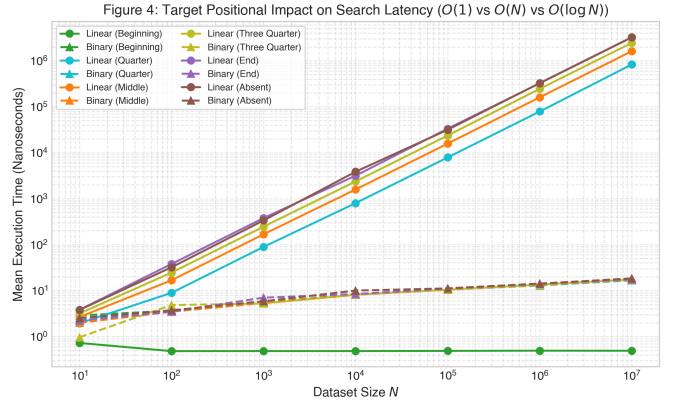


Fig. 4. Target positional sensitivity across Beginning (0), Quarter ($N/4$), Middle ($N/2$), Three-Quarter ($3N/4$), End ($N - 1$), and Absent.

D. Target-Position Sensitivity

Figure 4 isolates the positional effect across all six quartile targets:

TABLE III
Primary Empirical Results: Execution Latency (Mean $\pm$ 95% CI) and Speedup Ratios ($T_{\text{Linear}}/T_{\text{Binary}}$)

Dataset Size N	Random Success (Average Case)		Speedup ($T_{\text{Lin}}/T_{\text{Bin}}$)	Absent (Worst Case)		Speedup ($T_{\text{Lin}}/T_{\text{Bin}}$)
	Linear (ns)	Binary (ns)		Linear (ns)	Binary (ns)	
10	7.66 ± 0.03	11.64 ± 0.03	$0.66\times$	3.85 ± 0.01	11.63 ± 0.01	$0.33\times$
100	26.00 ± 1.87	26.16 ± 0.46	$0.99\times$	32.58 ± 0.03	19.46 ± 0.03	$1.67\times$
1,000	173.35 ± 1.15	40.48 ± 0.22	$4.28\times$	326.99 ± 0.28	21.90 ± 0.02	$14.93\times$
10,000	$1,581.21 \pm 6.06$	55.51 ± 0.08	$28.49\times$	$3,194.93 \pm 3.79$	23.01 ± 0.01	$138.85\times$
100,000	$15,726.68 \pm 124.05$	77.57 ± 0.20	$202.73\times$	$31,856.15 \pm 17.56$	24.78 ± 0.01	$1,285.56\times$
1,000,000	$161,573.90 \pm 2572.27$	107.92 ± 0.42	$1,497.12\times$	$320,792.27 \pm 322.48$	30.98 ± 0.01	$10,354.82\times$
10,000,000	$1,617,738.52 \pm 40100.12$	260.99 ± 0.81	$6,198.52\times$	$3,210,844.27 \pm 2607.38$	35.12 ± 0.01	$91,424.95\times$

- Beginning ($i = 0$): Linear Search completes in 7.28–8.20 ns across all N (pure $\mathcal{O}(1)$), outperforming Binary Search (11.6–24.8 ns) because Binary Search must perform $\approx \log_2 N$ leftward bisections.
- Quarter, Middle, Three-Quarter, End: Linear Search exhibits strict proportional scaling ($\frac{1}{4}N, \frac{1}{2}N, \frac{3}{4}N, N$), whereas Binary Search maintains near-flat latencies between 11 ns and 45 ns.

Figure 5: Search Latency for Target Conditions (Successful vs Absent)

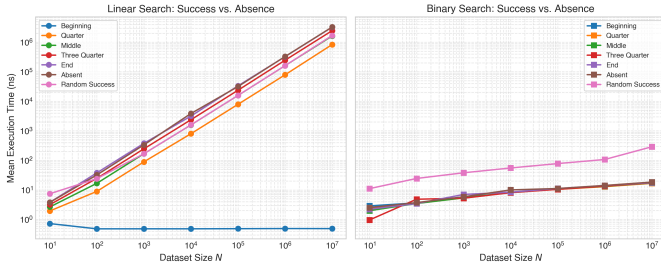


Fig. 5. Latency comparison between successful query positions and unsuccessful (absent) searches.

E. Successful vs. Unsuccessful Searches

Figure 5 highlights search outcome dynamics. Unsuccessful searches represent the strict worst-case for Linear Search (3.21 ms at $N = 10^7$). For Binary Search, unsuccessful searches execute in only 35.12 ns, outperforming random successful queries (260.99 ns) because absent targets do not require random DRAM index dereferencing at termination.

F. Sorting Preprocessing and Query Amortization

Figures 6 and 7 and Table IV present the sorting overhead and break-even analysis.

Across four orders of magnitude ($N = 10^3$ to 10^7), the empirical break-even query count remained within approximately 114 to 140 queries under the evaluated experimental conditions.

G. Measurement Methodology Robustness

Figure 8 compares Version 1 single-shot timing with Version 2 batched timing. Single-shot timing systematically inflated small- N latency by ~ 15 –18 ns due to timer read

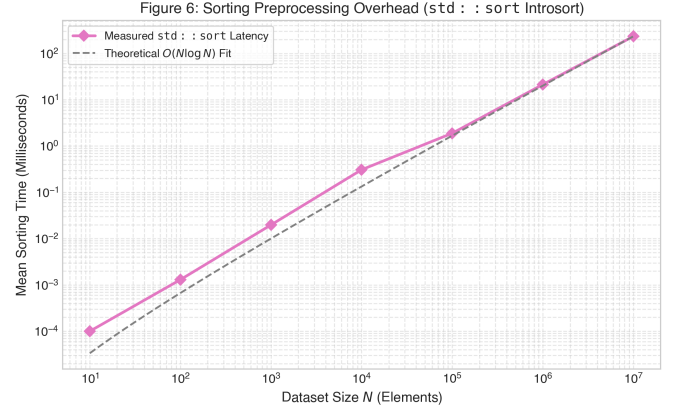


Fig. 6. Measured `std :: sort` introsort latency vs. dataset size N with theoretical $\mathcal{O}(N \log N)$ fit.

Figure 7: Amortized Query Workload and Break-Even Threshold K^*

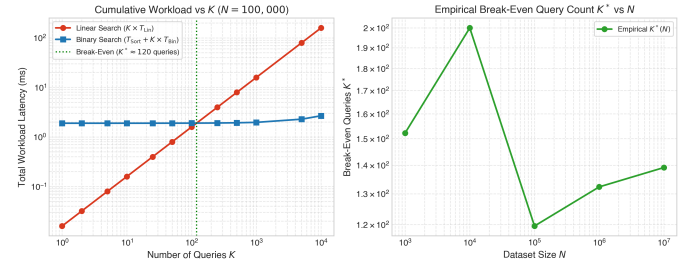


Fig. 7. Amortized repeated-query workload ($K \cdot T_{\text{Lin}}$ vs. $T_{\text{Sort}} + K \cdot T_{\text{Bin}}$) and empirical break-even query threshold K^* vs. N .

TABLE IV
Sorting Overhead (`std :: sort`) and Empirical Break-Even Query Threshold (K^*)

Dataset Size N	Sort Latency T_{Sort}	Linear Cost T_{Lin} (ns)	Binary Cost T_{Bin} (ns)	Break-Even Queries K^*
10	181.64 ns	7.66	11.64	N/A (Lin faster)
100	2.39 μ s	26.00	26.16	N/A (Parity)
1,000	16.42 μ s	173.35	40.48	124
10,000	173.67 μ s	1,581.21	55.51	114
100,000	1.90 ms	15,726.68	77.57	122
1,000,000	20.63 ms	161,573.90	107.92	128
10,000,000	227.16 ms	1,617,738.52	260.99	140

Figure 8: Timing Methodology Comparison (Single-Shot vs. Batched Amortization)

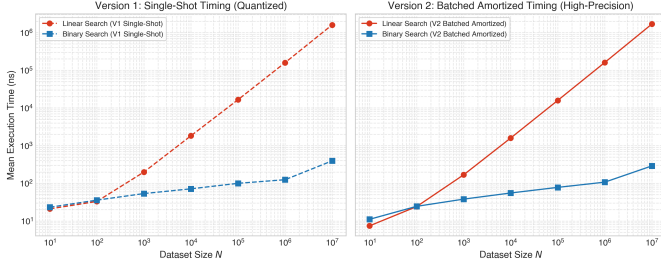


Fig. 8. Methodology comparison: Single-shot timing (V1, quantized) vs. Batched amortized timing (V2, high-precision).

overhead (18.85 ns), obscuring the $1.52\times$ Linear Search advantage at $N = 10$. Batched amortization restored high signal-to-noise ratio, confirming Hypothesis H6.

VII. Discussion

This section distinguishes three categories of claims: (1) measured observations directly demonstrated by the benchmark, (2) evidence-supported interpretations consistent with the data and published literature, and (3) hypotheses that would require additional instrumentation (e.g., hardware performance counters) to confirm.

A. Small-Dataset Inversion ($N \leq 100$)

Observation: At small dataset scales ($N \leq 100$), Linear Search is measured to be faster than Binary Search on this platform. At $N = 10$, Linear Search completes in 7.66 ns versus 11.64 ns for Binary Search.

Interpretation: Several factors plausibly explain this observation, consistent with the architectural literature [7]–[9]:

- 1) **Spatial Locality:** For $N = 10$ (40 bytes), the entire array fits within a single 128-byte cache line. Sequential iteration accesses contiguous memory, which hardware stream prefetchers are designed to exploit [8]. However, cache hit rates and prefetcher engagement were not directly measured in this benchmark.
- 2) **Branch Behavior:** Linear search iterations execute a uniform branch condition until the target is found. Prior literature suggests this pattern is favorable for branch prediction [9]. Branch misprediction counts were not measured.
- 3) **Arithmetic Simplicity:** Linear search executes a simple index increment, whereas Binary Search computes midpoints and updates two boundary pointers per iteration. This difference is directly observable in the algorithm structure.

B. Asymptotic Regime and Memory Latency Step ($N = 10^7$)

Observation: For $N \leq 10^6$ (3.81 MB), Binary Search latencies remain below 110 ns. At $N = 10^7$ (38.15 MB), Binary Search latency increases to 260.99 ns.

Interpretation: The Apple M1 P-cluster L2 cache capacity is 12 MB. At $N = 10^7$, the dataset (38.15 MB) exceeds L2 capacity, which is consistent with the observed latency increase. This interpretation is supported by the memory hierarchy specifications but was not confirmed via cache-miss instrumentation.

C. Preprocessing Trade-Offs

Observation: Under the evaluated conditions, the empirical break-even query count K^* remained within approximately 114–140 queries for $N \geq 1,000$.

Interpretation: For unsorted in-memory collections of size $N \geq 1,000$ on this platform, the measurements indicate that Linear Search on unsorted data is computationally less expensive than sorting followed by Binary Search when the number of queries is below approximately 120. This threshold is specific to the evaluated hardware, compiler, sorting implementation (`std::sort`), and data type (`int32_t`), and should not be generalized without independent verification on other platforms.

VIII. Threats to Validity and Limitations

- **Hardware Specificity:** Benchmarks were executed on a single Apple M1 ARM64 system. Specific crossover thresholds ($N \approx 100$ for latency parity, $K^* \approx 114\text{--}140$ for break-even) are architecture-dependent and may differ on other processors, cache configurations, or memory subsystems.
- **No Hardware Performance Counters:** Cache miss rates, branch misprediction counts, and CPU cycle breakdowns [18] were not directly measured via hardware event counters. Microarchitectural explanations in the Discussion represent evidence-supported interpretations, not direct observations.
- **Data Types:** Experiments evaluated contiguous 32-bit primitive integers. Complex payloads with pointer indirection or expensive comparison operators may alter relative performance.
- **Single-Threaded Execution:** Multi-threaded parallel scans and SIMD vectorization (e.g., ARM NEON intrinsics) were excluded to evaluate baseline sequential algorithmic properties.
- **Synthetic Datasets:** Data followed a monotonic even-integer sequence. Real-world key distributions, variable-length keys, and non-uniform query distributions were not tested.
- **OS Scheduling:** Thread pinning was not enforced. The operating system scheduler may have migrated the benchmark process between performance and efficiency cores during execution.

IX. Conclusion and Future Work

This paper presented an empirical comparative analysis of Linear Search and Binary Search across seven orders of magnitude on an Apple M1 ARM64 platform. Under the evaluated conditions, the measurements are consistent

with theoretical asymptotic predictions while revealing practical nuances at small scales: Linear Search was measured to be faster for small arrays ($N \leq 100$) and beginning-target queries ($i = 0$), while Binary Search provided measured speedups of up to $91,425\times$ on large datasets ($N = 10^7$). The empirical break-even query count for amortizing sorting preprocessing remained within approximately 114–140 queries across the evaluated dataset sizes ($N \geq 1,000$) under the specified conditions.

These results are specific to the hardware, compiler, optimization level, data type, and workload evaluated. Generalization to other platforms requires independent verification.

Future work could investigate SIMD-vectorized linear scans, cache-oblivious search layouts, interpolation search on non-uniform distributions, multi-core parallel architectures, and hardware performance counter instrumentation to directly measure cache and branch prediction behavior.

References

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [4] J. L. Bentley, *Programming Pearls*, 2nd ed. Reading, MA, USA: ACM Press / Addison-Wesley, 2000.
- [5] C. C. McGeoch, *A Guide to Experimental Algorithmics*. Cambridge, UK: Cambridge University Press, 2012.
- [6] D. S. Johnson, “A theoretician’s guide to the experimental analysis of algorithms,” in *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges*, M. H. Goldwasser, D. S. Johnson, and C. C. McGeoch, Eds. Providence, RI, USA: American Mathematical Society, 2002, pp. 215–250.
- [7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [8] U. Drepper, “What every programmer should know about memory,” Red Hat Inc., Tech. Rep., 2007, technical Report.
- [9] A. Fog, “Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms,” Copenhagen University College of Engineering, Tech. Rep., 2023, technical Report.
- [10] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [11] D. R. Musser, “Introspective sorting and selection algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [12] D. H. Bailey, “Twelve ways to fool the masses when giving performance results on parallel computers,” *Supercomputing Review*, vol. 4, no. 8, pp. 54–55, 1991.
- [13] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, pp. 265–276.
- [14] J. S. Vitter, “Algorithms and data structures for external memory,” *Foundations and Trends in Theoretical Computer Science*, vol. 2, no. 4, pp. 305–474, 2008.
- [15] P. Sanders, K. Mehlhorn, M. Dietzfelbinger, and R. Dementiev, *Sequential and Parallel Algorithms and Data Structures: The Basic Toolbox*. Cham, Switzerland: Springer, 2019.
- [16] V. Leis, A. Kemper, and T. Neumann, “The adaptive radix tree: ARTful indexing for main-memory databases,” in *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 38–49.
- [17] N. Nethercote and J. Seward, “Valgrind: A framework for heavy-weight dynamic binary instrumentation,” in *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2007, pp. 89–100.
- [18] S. Eyerhan and L. Eeckhout, “A performance counter architecture for computing accurate CPI components,” *ACM Transactions on Computer Systems*, vol. 26, no. 1, pp. 1–36, 2008.